

Incentive Fuzzer: Counterexample-Driven Adversarial Testing of Economic Rules

Aayush Kadam

Research Preview, version 0.1.0

September 2026

Abstract

Economic rules are executable incentive systems: eligibility tests, benefit schedules, procurement thresholds, reporting penalties, and strategic allocation rules turn observed states and actions into private and institutional outcomes. Small discontinuities or interaction effects can create profitable responses that are easy to miss in ordinary examples. Incentive Fuzzer is a research prototype that treats bounded economic rules as adversarially testable programs. It combines a restricted exact-decimal rule language, candidate search, deterministic replay, counterexample reduction, an independently authored exact-rational SMT backend, heterogeneous sensitivity analysis, finite strategic-game analysis, and regression testing of bounded repairs.

The artifact was developed through eight staged milestones with precommitments and hostile reviews. The semantic backend agreed with 500 of 500 generated fixed-input cases and all 12 frozen synthetic property statuses. In IF-Bench v0.1, blind combined search rediscovered the encoded structural pathology in five of six historical components, detected four of four positive holdouts, and produced no findings on ten negative controls. Boundary-only search matched combined recall while using fewer candidate evaluations, an important negative result. An SMT-only baseline supported 26 of 30 cases, detected 15 of 16 supported positives, produced no negative-control false positives, agreed with hidden ground truth on 25 of 26 supported cases, and replayed all 15 violating witnesses. These figures describe curated, partial, bounded encodings; they are not representative policy-auditing accuracy. External actions, valuations, and pathology mappings remain project-authored, and independent economist/domain review has not been completed.

1. Introduction

Software engineers test programs not only with expected examples but with inputs designed to break assumptions. Economic rules are also computational objects, but their failures are often strategic rather than syntactic. An applicant may alter reported income, a buyer may split transactions, a firm may reorganize activity, or several actors may coordinate around an aggregate threshold. The relevant question is not whether the rule computes its formula, but whether a feasible response changes private utility or designer outcomes in an unintended way.

Incentive Fuzzer asks: within a declared model, action space, and bounded domain, what state or strategic response violates an explicit property? Its answer is a witness containing the baseline state, action, resulting state, utility change, rule trace, classification, assumptions, and replay status. For supported fragments, a separate Z3 translation checks the same property using exact rational arithmetic. The project does not infer realistic behavior merely because an opportunity exists.

The work deliberately avoids a priority claim. Mechanism design, automated mechanism design, robust design, strategic classification, microsimulation, property-based testing, formal verification, and program repair all supply close antecedents. The research question is whether these disciplines can be composed into a reproducible evidence workflow for bounded economic rules.

Contributions

1. **IncentiveSpec v0.1**, a restricted rule language with typed units, exact decimal arithmetic, simultaneous transitions, ordered rules, separate private utility and designer outcomes, canonical serialization, and structured traces.
2. **Replay-first adversarial search**, including exhaustive, boundary, seeded-property, grid, and combined methods with structural deduplication and bounded economic counterexample reduction.
3. **An independently implemented formal backend** for profitable-deviation existence in a QF_LIRA fragment, with exact witness replay through the runtime.
4. **Sensitivity and interaction layers** for synthetic heterogeneous populations and exact finite pure-strategy games.
5. **Bounded repair regression**, in which candidates are re-attacked across applicable search, formal, population, strategic, and mutation layers.
6. **IF-Bench v0.1**, a blind, versioned suite with internal cases, externally sourced components, holdouts, negative controls, provenance, separated labels, and preserved misses.

2. Motivation and Threat Model

An incentive failure is meaningful only relative to an actor, information set, feasible action, timing model, utility function, and designer property. Incentive Fuzzer therefore separates intended response, adaptation, manipulation, and ambiguity rather than calling every profitable action an exploit. The DOL safe-harbor matching stress case illustrates the point: increasing a contribution to obtain an employer match is profitable under the encoded valuation, but that response is intended and is explicitly labeled as such.

The current individual semantics are one-agent, one-period, deterministic, and finite-domain. Actions are declared rather than inferred. “No violation found” means only that no witness was found within domain D under assumptions A and configuration C. FORMALLY_SATISFIED_WITHIN_DOMAIN is similarly scoped: it says that the encoded negation is unsatisfiable in the supplied fragment and domain, not that an institution is safe in the world.

3. Related Work

Classical mechanism design studies rules under strategic response; automated and computer-aided mechanism design search mechanism spaces subject to incentive constraints. Robust mechanism design studies uncertainty in types or objectives. Strategic classification studies agents who manipulate features in response to decision rules. These literatures prevent any claim that strategic response analysis or computational mechanism design originates here.

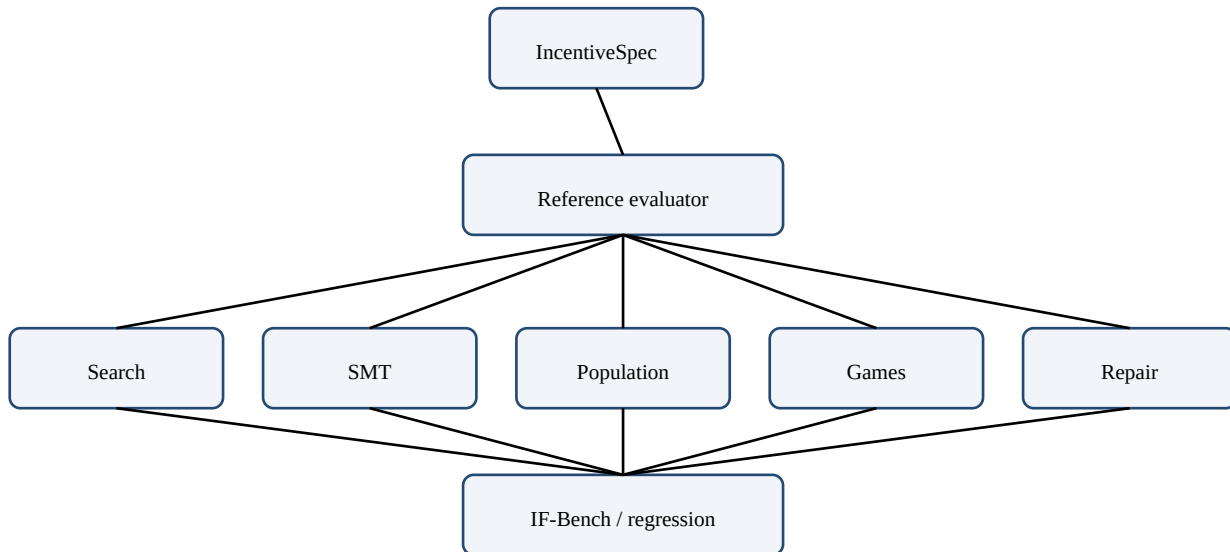
Tax-benefit microsimulation and rules-as-code systems provide executable policy models. OpenFisca documents a static microsimulation core, while PolicyEngine includes selected behavioral-response models. Tax bunching and notch research studies behavioral responses around nonlinear schedules. These systems and literatures are more mature in their respective domains than Incentive Fuzzer.

Property-based testing supplies generated inputs and shrinking; SMT solvers supply models or scoped unsatisfiability for encoded formulas; formal mechanism-verification research translates strategic properties into proof obligations; program repair uses counterexamples and regression suites to constrain candidate fixes. Incentive Fuzzer studies their composition as an audit workflow with economic threat models, exact replay, domain-aware reduction, robustness,

strategic interaction, and repair re-attack. The integration is the contribution candidate; its individual components are not claimed as novel.

4. System Architecture

Figure 1. Evidence architecture



The core pipeline is:

```
Rule -> executable semantics -> adversarial search -> counterexample
    -> replay/formal confirmation -> population robustness
    -> strategic interaction -> repair -> regression re-attack
    -> external benchmark evidence
```

The trusted runtime resides in `core.py`. Search, formal verification, population analysis, games, repair, and benchmark evaluation are separate layers. Search candidates and SMT witnesses must satisfy the runtime semantics before becoming positive evidence. The formal backend shares parsing and schema validation with the runtime but independently implements decimal-to-rational mapping, AST translation, transitions, ordered rules, costs, utility, and property negation.

5. IncentiveSpec and Execution Semantics

IncentiveSpec v0.1 represents a bounded institution as state, action, transition, rule, cost, utility, designer outcome, parameter, and property objects. Source YAML is serialization, not executable code. Parsing produces frozen typed objects from an allow-listed expression AST. Unsupported operators, units, variables, transitions, and bounds fail closed.

Money is represented with exact `Decimal` values parsed from text. The language supports addition, subtraction, scalar/rate multiplication, comparisons, minimum, maximum, and total conditionals. All action-transition right-hand sides read the same pre-action state, giving simultaneous semantics. Rules evaluate in declaration order. Agent utility is distinct from fiscal or administrative outcomes.

M1 validated nine manual cases and 65 tests. Its principal limitation was the absence of an independent evaluator; M3 later reduced, but did not eliminate, that risk. The parser remains shared, statutory rounding modes are absent, and the

language does not cover multiple periods or arbitrary agent interaction.

6. Adversarial Search and Reduction

M2 introduced explicit finite domains and budgets. Exhaustive enumeration supplies ground truth only within a declared finite domain. Boundary search extracts simple attribute-versus-constant comparisons and constructs nearby values. Seeded property search emphasizes extrema and interior points; grid search uses regular representatives; combined search prioritizes boundary candidates before generic candidates.

Every candidate is evaluated at baseline and after action through the M1 oracle. Findings contain structural equivalence keys, IF-CWE classifications, traces, and assumptions. Reduction searches the supplied finite neighborhood under a lexicographic objective. “Minimal” therefore means minimal under that objective and domain, not globally minimal.

On the frozen 12-case M2 suite, boundary search recovered all exhaustive equivalence classes with 4,812 evaluations versus 45,144 for exhaustive search, 89.3 percent fewer. The suite is self-authored and threshold-heavy, so this is not a general scaling claim. Exhaustive evaluation also disproved two initially planted safe labels; the labels were corrected rather than the findings suppressed.

7. Formal Verification

The M3 backend uses Z3 over quantifier-free linear integer/real arithmetic with `ite`, exact comparisons, minimum, maximum, conditionals, simultaneous transitions, and ordered rules. It searches for a feasible action whose post-action utility is strictly greater than baseline utility. SAT witnesses are decoded and replayed exactly; replay disagreement is a first-class failure. UNSAT is bound to a property and domain hash.

The backend agreed on 500 of 500 generated fixed-input cases, five of five canonical property statuses, and 12 of 12 frozen benchmark statuses. It also handled phase-out min/max expressions that M2 boundary extraction did not recognize. Only profitable-deviation existence is formally encoded; four other M1 property families remain enumeration-only.

M7.5 completed a label-blind SMT-only baseline. Twenty-six of 30 IF-Bench cases were supported. The solver returned 15 formal violations and 11 scoped satisfactions, with no timeout, unknown, or backend disagreement. It detected 15 of 16 supported positives, produced zero false positives on ten supported negative controls, agreed with hidden ground truth on 25 of 26 supported cases, agreed with combined search on all 26, and replayed 15 of 15 violating witnesses. Transaction splitting and three composition cases remain unsupported. The sole ground-truth mismatch is the preserved Arizona proxy.

8. Robustness Analysis

M4 separates profitable opportunity from behavioral response. Synthetic agent types carry fixed state, weight, friction, cost multiplier, action availability, hurdle, and provenance. B0 exhausts declared candidates, B1 searches a restricted ordered subset, and B2 discovers all opportunities but adopts only above a declared hurdle.

In the registered scholarship grid, B0 profitable share was 0.515625 and maximum adjusted gain was 99,999. The registered phase-out had zero profitable cases in its own finite grid. A weighted synthetic population had exact profitable share 0.9; seeded Monte Carlo estimates approached that value as sample size increased. These results

demonstrate deterministic aggregation and sensitivity, not empirical recipient behavior or uncertainty about model misspecification.

9. Strategic Interaction

M5 adds an explicit finite-game wrapper rather than silently changing IncentiveSpec. It computes exact payoff profiles, best-response correspondences with ties preserved, complete pure-strategy Nash sets, deviation witnesses, declared best-response dynamics, cycles, and basins.

The flagship two-player game has payoff cells $(H, H) = (10, 10)$, $(H, M) = (10, 8)$, $(M, H) = (8, 10)$, and $(M, M) = (12, 12)$. Manipulation has gain -2 against an honest rival and +2 against a manipulating rival. Both (H, H) and (M, M) are pure equilibria. An independent-agent calculation would report no manipulation, missing the interaction-supported equilibrium.

This is a deliberately constructed complete-information game. It demonstrates semantic capability, not empirical equilibrium selection. Mixed, Bayesian, sequential, coalition, learning, and continuous-action models remain outside scope.

10. Counterexample-Driven Repair

M6 defines bounded repair spaces, constraints, objectives, regression adapters, mutation neighborhoods, and Pareto comparison. Candidate search is transparent finite-grid enumeration plus one generic hard-cutoff-to-linear-phase-out transform. It is not general program synthesis.

Four scholarship phase-outs eliminated the registered single-agent finding and synthetic population exposure while increasing fiscal cost under declared states. Procurement action-cost multipliers 19 and 20 passed the registered gates. No constraint-feasible stacked-program award pair fully removed its vulnerability. Six strategic-game parameter combinations preserved (H, H) while removing (M, M) .

The most important repair result is negative. A deliberately coupled enforcement-rebate candidate changed an isolated gain from +1 to 0 but increased an interaction bonus enough to create (M, M) . The gate classified it REPAIR_INDUCED_VULNERABILITY and REPAIR_FAIL. A passing candidate remains safe only relative to the declared regression suite.

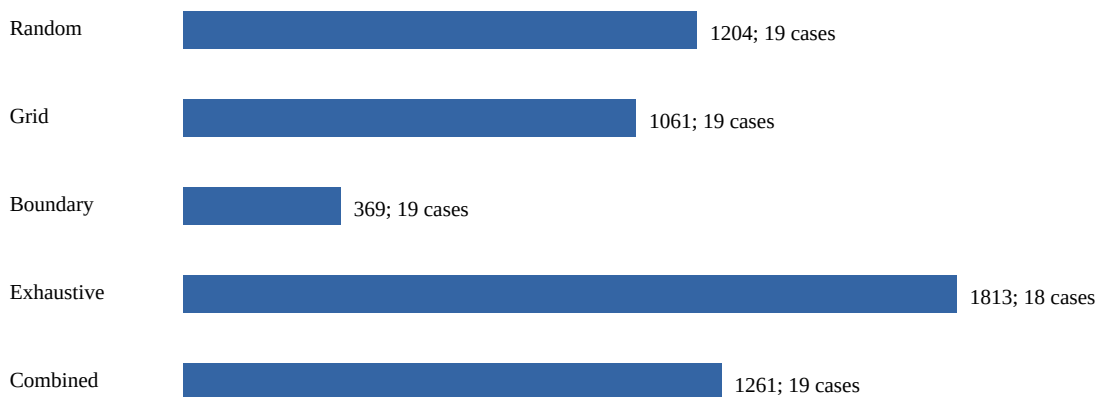
11. IF-Bench v0.1

IF-Bench contains 30 bounded cases: 12 inherited internal synthetic cases, eight additional internal/control cases, ten external components, six historical components, ten negative controls, and six holdouts. Counts overlap. Runtime files and hidden labels are separated, and integrity tests reject label fields from runtime input.

External components include the ACA premium-tax-credit cutoff, FAR micro-purchase splitting, SNAP gross-income screening, state Medicaid/CHIP eligibility components, the Social Security retirement earnings test, and the EITC phase-out. Every external encoding is partial. Official sources establish rule components; they do not independently validate project-authored actions, normalized values, or pathology mappings.

12. External Evaluation

Figure 2. IF-Bench detections and candidate evaluations



SMT-only omitted: solve time is not a candidate-evaluation count.

Combined search rediscovered five of six historical encoded components: ACA, FAR, SNAP, Alabama Medicaid, and Pennsylvania CHIP. It missed the Arizona child-transition case. Positive holdout detection was four of four; neither of two negative holdouts produced a finding; all ten negative controls were clean. Six external violations and three external scoped non-violations received formal confirmation where supported.

The central baseline result is unfavorable to search complexity. Random, grid, boundary-only, bounded exhaustive, and combined methods detected 19, 19, 19, 18, and 19 cases, respectively. Their candidate evaluations were 1,204, 1,061, 369, 1,813, and 1,261. Boundary-only therefore matched combined recall with substantially fewer evaluations. The integrated workflow adds blindness, replay, reduction, formal evidence, and regression structure; it does not add recall on this suite.

The Arizona miss is preserved. Official thresholds did not justify treating Medicaid and CHIP coverage as additive cash awards. The honest smooth proxy therefore did not exhibit the expected composition pathology. Retuning the case to force success would have weakened the benchmark.

Threat-model dependence is equally important. Removing downward-adjustment or transaction-splitting actions eliminated every external finding. Thus external rules do not make the action model external.

13. Supplementary Structural Stress Tests

M7.5 added three separate stress cases without changing IF-Bench. A continuous IRS single-filer progressive schedule was formally satisfied within its declared domain. An IRS dependent-care credit percentage staircase produced a replayed local gain under a declared income-forgoing action; this is analytically derived, not documented manipulation. A DOL safe-harbor 401(k) tiered/capped match produced a replayed positive response explicitly classified as intended behavior. Three cases do not establish broad non-threshold coverage.

14. Reproducibility and Artifact Design

The repository pins Python dependencies, records seeds, preserves experiment tables and manifests, and provides quick, research, and full reproduction modes. Public CLI commands emit exact statuses rather than safe/unsafe

labels. The clean-install protocol creates an isolated Python 3.12 environment, installs the package and test extras, imports the API, runs tests, exercises CLI commands and demos, validates benchmarks, and regenerates headline tables and figures.

The external-review package gives economists and domain experts compact case cards, explicit questions about fidelity, actions, valuation, and mapping, and a sealed mini-holdout format. Runtime evaluation does not load hidden labels. This infrastructure is not evidence that review occurred.

15. Limitations

The strongest limitation is authorship dependence. External rules are authoritative, but encodings, action spaces, normalized values, and classifications are project-authored. Independent economist/domain review remains outstanding. IF-Bench is small, US-heavy, scalar, and threshold-heavy. It cannot support representative accuracy estimates.

The semantics are deterministic, bounded, and mostly one-agent/one-period. Formal verification covers one property and shares parsing with the runtime. Population weights and behavior are synthetic. Strategic analysis is finite, pure-strategy, simultaneous, and complete-information. Repair spaces and objectives are bounded and normative. No empirical prevalence, legality, welfare, causal, program-completeness, or policy-optimality claim is supported.

16. Discussion

The evidence suggests that disciplined software-testing concepts can improve the auditability of economic-rule analysis. Explicit domains prevent universal language; replay separates candidate generation from truth; differential verification exposes semantic disagreement; preserved misses discourage benchmark gaming; multi-layer regression shows how a local fix can fail elsewhere.

The same evidence limits the ambition. On IF-Bench v0.1, a simple boundary method is the strongest practical detector. Manual modeling remains the decisive scientific input. The project is most credible as an inspectable workflow for formulating and challenging bounded models, not as an autonomous auditor.

17. Conclusion

Incentive Fuzzer turns a declared economic rule into an auditable chain of evidence: executable semantics, adversarial candidates, replayed counterexamples, formal cross-checks, sensitivity analysis, strategic interaction, repair re-attack, and blind benchmark evaluation. The artifact preserves both positive and negative results, including the Arizona miss, boundary-only's strong performance, and action-space dependence.

The repository is ready as a research preview and external-review package. It is not externally validated for policy use. The next scientific step is not a stronger claim; it is independent review of encodings, actions, valuations, and pathology mappings, followed by a genuinely external sealed holdout.

References

The complete source register is maintained in `bibliography/SOURCES.md`. Primary rule sources are also frozen in benchmark source registers. Key conceptual sources include Hurwicz and Reiter on mechanism design; Conitzer and

Sandholm on automated mechanism design; robust mechanism-design literature; Hardt et al. on strategic classification; OpenFisca and PolicyEngine documentation; MacIver et al. on Hypothesis; de Moura and Bjorner on Z3; computer-aided mechanism-design and verification work; tax bunching research; and counterexample-guided program-repair literature. External rule sources include the Internal Revenue Service, Acquisition.gov, federal oversight reports, CMS/Medicaid, the Social Security Administration, and the U.S. Department of Labor.

Appendix A. Frozen headline results

The authoritative table is `research/m8_entry/HEADLINE_TABLES.md`. Quantitative claims in this manuscript are mapped to exact artifacts and commits in `research/M8_CLAIM_AUDIT.md`.

Appendix B. Reproduction

Install Python 3.12, create an isolated environment, install `. [test]`, and run `python scripts/reproduce.py full`. The command runs the complete test suite, validates M7 and M7.5 logical results, regenerates final tables and figures, executes the three public demos, and writes `reproducibility/manifest.json`.

Appendix C. External review

Independent reviewers should begin with `external_review/README.md`. Review findings must be recorded without overwriting frozen labels. Material corrections require a versioned benchmark update. The sealed-holdout template keeps runtime inputs and hidden labels separate; the project does not call project-authored cases independent.